

MEMARCH: Análisis Comparativo de la Ejecución Recursiva e Iterativa a Nivel Arquitectónico mediante C e Integración ASM x86-64 en plataforma WSL Linux 2.4.7 Ubuntu

Manuel Hernández¹, Daniel Saucedo León²

Doctorado en Sistemas Computacionales

— Entregable: Semana 2 — mayo 2026

Abstract— This paper presents MEMARCH, an applied research project that experimentally analyzes the architectural behavior of recursive and iterative algorithm implementations at the x86-64 machine level. The central hypothesis states that semantic equivalence between recursion and iteration does not imply architectural equivalence: the two paradigms produce measurably different behaviors with respect to activation stack depth, register usage, instruction count, and cache pressure. The project integrates pure C with hand-written x86-64 assembly (NASM, Intel syntax) to implement and instrument canonical algorithms —factorial and Fibonacci— under both execution paradigms, capturing hardware-level evidence through compiler-generated disassembly analysis and performance counter instrumentation via Linux 2.4.7 perf. Findings are interpreted against the Von Neumann execution model, the System V AMD64 ABI calling convention, and computer memory hierarchy theory. Results are expected to quantify the $O(n)$ versus $O(1)$ stack growth disparity and its practical implications for compiler optimization flags, tail-call elimination, and high-performance systems design.

Keywords— Von Neumann architecture, x86-64, recursion, iteration, stack frame, ABI, ASM, pipeline, cache hierarchy, tail-call optimization.

I. INTRODUCCIÓN

La equivalencia entre recursión e iteración es un resultado clásico de la teoría de la computabilidad: todo algoritmo expresable de forma recursiva puede reformularse de manera iterativa y viceversa, sin pérdida de expresividad computacional [1]. Sin embargo, esta equivalencia es semántica, no arquitectónica. Es razonable plantear como hipótesis de trabajo que, cuando dos programas semánticamente equivalentes se ejecutan sobre hardware real —específicamente sobre un procesador x86-64 basado en el modelo de Von Neumann—, podrían producir trazas de ejecución, patrones de uso de memoria y tiempos de respuesta diferenciables. La magnitud, dirección y reproducibilidad de tales diferencias constituyen precisamente el objeto de validación experimental del presente proyecto, por lo que no se asumen a priori sino que se someterán a medición instrumentada. Este modelo es referente de compatibilidad en el hardware.

La distinción emerge del modo en que el hardware implementa la abstracción de subrutina. Una llamada recursiva implica la construcción de un nuevo marco de activación en el stack, el almacenamiento del puntero de retorno, la preservación de registros callee-saved y la transferencia de control mediante la instrucción CALL. En contraste, una implementación iterativa del mismo algoritmo puede, bajo condiciones ideales, prescindir de todo uso de stack adicional y operar exclusivamente en registros de propósito general. La afirmación de un "tráfico de memoria reducido a cero" y de "optimización de tiempos" debe entenderse como una expectativa teórica sujeta a las

decisiones que tome el compilador (registros disponibles, spills al stack por presión de registros, alineación de pila por el ABI) y al comportamiento microarquitectónico real del procesador empleado en las pruebas. Estas condiciones se verificarán mediante inspección del ensamblador generado y contadores de hardware antes de afirmar resultados concluyentes.

El presente proyecto, denominado MEMARCH (Memory + Architecture), propone instrumentar experimentalmente esta distinción con demostración de código puro mediante la implementación manual de rutinas en lenguaje ensamblador x86-64 (NASM, sintaxis Intel) integradas con programas en C puro; se medirán, documentarán y analizarán las diferencias arquitectónicas entre la ejecución recursiva e iterativa de algoritmos canónicos (por ejemplo, serie Fibonacci con ciclos y factorial de un número con recursividad). El entorno experimental opera sobre Windows 11 x86-64 y utiliza herramientas de análisis como GCC corriendo en WSL Linux 2.4.7 con diferentes niveles de optimización, objdump para inspección de código máquina y perf para captura de contadores de hardware.

La importancia de este análisis trasciende lo didáctico. En sistemas de alto rendimiento, la elección entre recursión e iteración —y la comprensión de sus consecuencias a nivel de microarquitectura— podría asociarse, según la literatura, con efectos sobre el consumo de stack, la tasa de cache-miss, la predicción de saltos y el throughput del pipeline del procesador. La validación cuantitativa de estos efectos en el entorno experimental concreto de MEMARCH constituye uno de los aportes que el proyecto buscará verificar y reportar. Para los diseñadores de compiladores, los

ingenieros de sistemas operativos y los investigadores en arquitectura de computadoras, esta distinción es operativamente significativa.

II. PLANTEAMIENTO DEL PROBLEMA

En la enseñanza y práctica convencional de algoritmos, la recursión y la iteración se presentan frecuentemente como estrategias intercambiables, base obligada en el estudio de estructuras de datos y algoritmos; en programación orientada a objetos, son tópicos avanzados seleccionables por criterios de legibilidad o estilo de programación. Esta perspectiva, si bien pedagógicamente útil, omite una dimensión crítica para el ingeniero de sistemas: el comportamiento de cada paradigma en el sustrato de hardware sobre el que ejecuta.

A. Pregunta de Investigación

¿En qué medida y de qué manera difieren, a nivel de arquitectura x86-64, la ejecución recursiva y la ejecución iterativa de un mismo algoritmo, y cómo pueden estas diferencias cuantificarse experimentalmente mediante instrumentación directa del código ensamblador?

B. Hipótesis Central

HI: La implementación recursiva de un algoritmo genera, en arquitectura x86-64, un consumo de stack de complejidad $O(n)$ en profundidad de recursión, mientras que su equivalente iterativo opera con complejidad $O(1)$ en uso de stack, diferencia medible mediante análisis de registros *rsp/rbp* e instrumentación de contadores de hardware.

C. Alcance

El proyecto se delimita al análisis de algoritmos con estructura de recursión lineal simple (factorial) y recursión arbórea con memoización (Fibonacci), implementados en ensamblador x86-64 NASM bajo el ABI System V AMD64, integrados con programas en C puro compilados con GCC en plataforma WSL Linux 2.4.7 distribución Ubuntu. No se aborda recursión mutua, recursión con continuaciones ni arquitecturas ARM o RISC-V por la carencia del hardware.

III. JUSTIFICACIÓN TÉCNICA

La justificación de MEMARCH descansa sobre tres ejes convergentes:

1) Relevancia arquitectónica:

El modelo de Von Neumann establece que la CPU y la memoria son entidades separadas conectadas por un bus. Toda llamada de función —y en particular toda llamada recursiva— implica operaciones sobre ese bus: escritura del puntero de retorno en stack (memoria), lectura del nuevo frame de instrucciones (memoria), y eventual escritura de valores de retorno en registros. Cuantificar el volumen de estas operaciones en función del paradigma de ejecución es una contribución directa a la comprensión del cuello de

botella de Von Neumann en contextos algorítmicos concretos.

2) Pertinencia para sistemas de alto rendimiento:

En compiladores optimizantes (GCC -O2, LLVM), la eliminación de llamadas en cola (tail-call elimination, TCE) convierte automáticamente ciertas funciones recursivas en bucles iterativos. Comprender la condición arquitectónica que hace posible esta transformación —la ausencia de trabajo pendiente tras la llamada recursiva— requiere entender el protocolo de llamada a nivel de instrucciones de máquina. MEMARCH proporciona esta comprensión de forma experimental y verificable.

3) Aportación:

Esta investigación en sistemas computacionales demanda la capacidad de construir evidencia experimental reproducible en el nivel de abstracción más bajo accesible al software: el código máquina. MEMARCH desarrolla esta competencia de forma directa, al requerir que el equipo diseñe, implemente, compile, desensamble e instrumente código en los tres niveles del modelo —algoritmo, lenguaje de alto nivel, ensamblador— y correlacione los resultados con teoría arquitectónica documentada; aunque por restricciones de tiempo será difícil alcanzar mediciones exhaustivas y propuestas de alta gama, se obtendrá una demostración clara y sostenida del objeto de la asignatura.

IV. OBJETIVOS

A. Objetivo General

Diseñar, implementar y documentar una prueba de concepto funcional que analice experimentalmente las diferencias arquitectónicas entre la ejecución recursiva e iterativa en procesadores x86-64, mediante la integración de lenguaje ensamblador (NASM) con programas en C puro, sustentando los hallazgos en el modelo de Von Neumann, la jerarquía de memoria y el ABI System V AMD64.

B. Objetivos Específicos

- Fundamentar teóricamente el modelo de ejecución de Von Neumann, el protocolo de llamada ABI x86-64 (Application Binary Interface) y la jerarquía de memoria como marco de análisis para el comportamiento de subrutinas recursivas e iterativas.
- Implementar en ensamblador x86-64 (NASM, sintaxis Intel) tres rutinas instrumentadas: factorial recursivo, factorial iterativo y Fibonacci con memoización (almacenar resultados de funciones costosas en cache), documentando el uso de registros, instrucciones de control y operaciones de stack.
- Integrar las rutinas ASM con un programa principal en C puro que ejecute benchmarking con resolución nanosegundo y capture diferencias en profundidad de stack, tiempo de ejecución e instrucciones emitidas.
- Analizar comparativamente el código ensamblador generado por GCC bajo niveles de optimización O0 y O2, contrastando con las rutinas escritas a mano para

identificar transformaciones de tail-call y eliminación de frames.

- Documentar los resultados en un reporte de investigación con estructura y formato IEEE/ASME, incluyendo tablas experimentales, análisis cuantitativo y argumentación técnica sustentada en referencias especializadas.

V. MARCO TEÓRICO

A. Arquitectura de Computadoras

La arquitectura de computadoras es la disciplina que estudia la organización y el comportamiento funcional de los sistemas de cómputo desde la perspectiva del programador de sistemas y del diseñador de hardware. Su objeto de estudio central es la interfaz entre el software y el hardware: el conjunto de instrucciones (Instruction Set Architecture, ISA), la organización interna del procesador, la jerarquía de memoria y los mecanismos de entrada/salida [2].

Patterson y Hennessy [2] distinguen tres niveles de abstracción: la arquitectura (ISA, visible al programador), la microarquitectura (implementación del ISA, pipeline, predicción de saltos) y la implementación física (circuitos, tecnología de fabricación). Para el propósito de MEMARCH, el análisis opera principalmente en los dos primeros niveles: se observa el comportamiento del ISA x86-64 mediante código ensamblador, y se infiere el comportamiento microarquitectónico mediante contadores de hardware accesibles vía el subsistema perf de Linux, ya que Windows resulta limitado y restrictivo respecto al uso de un sistema GNU/Linux en el ámbito de la profundidad de estas pruebas.

La arquitectura x86-64 (también denominada AMD64 o Intel 64) es una extensión de 64 bits de la arquitectura x86 original. Define 16 registros de propósito general de 64 bits (RAX, RBX, RCX, RDX, RSI, RDI, RSP, RBP, R8–R15), un puntero de instrucción (RIP), registros de flags (RFLAGS) y un conjunto de instrucciones retrocompatible con el código de 32 y 16 bits. El ISA x86-64 es de tipo CISC (Complex Instruction Set Computer), con instrucciones de longitud variable y modos de direccionamiento altamente flexibles [3].

B. Modelo de Von Neumann

El modelo de Von Neumann, formulado por John von Neumann en 1945 en el informe EDVAC [4], establece los principios organizativos fundamentales que han dominado el diseño de computadoras durante más de ocho décadas. Sus postulados esenciales son:

- Programa almacenado: las instrucciones del programa se almacenan en la misma memoria que los datos, codificadas en forma binaria.
- Secuencialidad: la CPU ejecuta instrucciones en orden secuencial, determinado por un registro contador de programa (Program Counter, PC o RIP en x86-64).

- Bus único: un único canal de comunicación (bus) interconecta la CPU con la memoria principal, por el cual transitan tanto instrucciones como datos.
- Separación funcional: la unidad de procesamiento (CPU) y la unidad de almacenamiento (memoria) son entidades físicamente distintas.

El corolario arquitectónico más importante del modelo, para los propósitos de MEMARCH, es el denominado cuello de botella de Von Neumann (Von Neumann bottleneck), señalado por John Backus en su conferencia Turing de 1977 [5]: la velocidad con que la CPU puede procesar información es estructuralmente limitada por el ancho de banda del bus que la conecta con la memoria. Cualquier operación que requiera accesos frecuentes a memoria —como la construcción y destrucción de marcos de activación en cada llamada recursiva— opera bajo esta restricción.

En el contexto de MEMARCH, el modelo de Von Neumann provee el marco interpretativo para entender por qué la recursión profunda tendería a ser arquitectónicamente más costosa que la iteración: cada nivel de recursión impone, al menos en su forma canónica, dos accesos al bus (escritura del frame de activación, lectura del puntero de retorno), mientras que la implementación iterativa equivalente podría reducir o eliminar dichos accesos si opera dentro del banco de registros, evitando el tráfico al bus principal. Estas afirmaciones son consistentes con el modelo Von Neumann y con la literatura, y se validarán empíricamente mediante contadores de hardware antes de presentarse como resultados del proyecto. He aquí el principio de evitar salir al bus.

C. Procesador

El procesador (CPU) es el componente que implementa el ciclo de ejecución de instrucciones definido por el modelo de Von Neumann. En su forma clásica, este ciclo comprende tres fases: Fetch (lectura de la instrucción desde memoria), Decode (decodificación del opcode y operandos) y Execute (ejecución de la operación aritmético-lógica o de control de flujo). Los procesadores modernos implementan variantes altamente optimizadas de este ciclo mediante pipelining, ejecución fuera de orden (out-of-order execution) y ejecución especulativa, referente de esta investigación. En este modelo el procesador de prueba será:

Componente	Valor
Host operativo	Microsoft Windows 11 Pro · build 10.0.26200 (Compilación 26200)
Fabricante del SO	Microsoft Corporation
Subsistema	WSL versión 2.7.3.0
Kernel WSL2	Linux 6.6.114.1-microsoft-standard-WSL2 (SMP PREEMPT_DYNAMIC)
Compilador del kernel	GCC 13.2.0 con GNU ld 2.41
Distribución guest	Ubuntu 26.04 LTS (Resolute Raccoon)
Tipo de virtualización	Hyper-V (full virtualization, VT-x con EPT/VPID)
Hypervisor vendor	Microsoft
WSLg	1.0.73 (soporte gráfico Wayland/X11)
Direct3D	1.611.1-81528511
DXCore	10.0.26100.1-240331-1435.ge-release
MSRDC	1.2.6676

Tabla C1. Subsistema WSL2 y distribución guest Ubuntu utilizada como sandbox MEMARCH. El kernel 6.6.114.1 con preempción dinámica habilita `perf_event_open()` y los contadores del PMU del procesador.

Para el análisis de MEMARCH, los elementos del procesador directamente relevantes son:

1) Registros de propósito general:

En x86-64, los 16 registros de 64 bits constituyen el almacenamiento más rápido disponible (latencia ~0 ciclos). El ABI System V AMD64 clasifica estos registros en caller-saved (RAX, RCX, RDX, RSI, RDI, R8–R11, cuyos valores puede destruir el procedimiento llamado) y callee-saved (RBX, RBP, R12–R15, que el procedimiento llamado debe preservar si los usa) [6]. Se espera, según la literatura sobre el ABI, que la recursión tienda a incrementar la presión sobre los registros callee-saved y a forzar su almacenamiento en stack; el grado real de esta presión en nuestras rutinas se determinará por inspección del ASM generado y, donde aplique, por instrumentación.

2) Unidad de predicción de saltos:

El procesador mantiene un Buffer de Predicción de Saltos (Branch Prediction Buffer, BPB) que anticipa el destino de instrucciones de salto condicional. Las instrucciones CALL y RET de la recursión interactúan con el Return Address Stack (RAS), un búfer dedicado de ~16–32 entradas que predice los destinos de instrucción RET. Una recursión más profunda que la capacidad del RAS degrada la predicción y genera penalizaciones de pipeline.

3) Unidad aritmético-lógica (ALU):

Ejecuta las operaciones de cómputo efectivo (IMUL, ADD, SUB). Bajo la premisa de equivalencia algorítmica, se espera que ambos paradigmas —recursivo e iterativo— ejecuten un número comparable de operaciones aritméticas en la ALU para un mismo n ; sin embargo, esta hipótesis se contrastará contando instrucciones retiradas mediante Linux perf, dado que las optimizaciones del compilador pueden alterar el conteo efectivo. Se anticipa que, si efectivamente la carga aritmética es similar, las diferencias de rendimiento observadas se atribuirían principalmente al overhead de control de flujo y al acceso a memoria, atribución que también deberá ser verificada por instrumentación.

D. Memoria

La jerarquía de memoria en sistemas x86-64 modernos comprende múltiples niveles organizados por latencia y capacidad: registros (< 1 ns), caché L1 (4–5 ciclos, 32–64 KB por núcleo), caché L2 (12–15 ciclos, 256 KB–1 MB), caché L3 (30–50 ciclos, 4–64 MB compartidos), DRAM principal (60–100 ns, GBs) y almacenamiento secundario [2].

Nivel	Por core	Total	Asociatividad	Línea	Tipo	Latencia ref.
L1d	32 KB	128 KB (4×)	8-way	64 B	Datos	4–5 ciclos
L1i	32 KB	128 KB (4×)	8-way	64 B	Instrucciones	4–5 ciclos
L2	256 KB	1 MB (4×)	4-way	64 B	Unified	12–14 ciclos
L3	—	6 MB (compartida)	12-way	64 B	Unified	40–50 ciclos

Tabla D.1. Jerarquía de caché del Intel Core i5-10310U. Las latencias de referencia son las documentadas por Intel para la familia Skylake/Comet Lake y deben validarse experimentalmente con TB-09 (pointer-chasing) sobre este equipo específico.

En el contexto de llamadas a función, el elemento de memoria crítico es el stack (pila de llamadas). En x86-64, el stack crece hacia direcciones decrecientes a partir de un límite superior asignado por el sistema operativo. El registro RSP (Stack Pointer) apunta al tope actual del stack. La instrucción CALL decrementa RSP en 8 bytes y escribe la dirección de retorno; la instrucción RET lee esa dirección y restaura RSP. Cada marco de activación (stack frame) puede contener: la dirección de retorno (8 bytes), el RBP anterior guardado (8 bytes), variables locales y parámetros adicionales que no caben en registros.

La implicación para MEMARCH es directa: una función recursiva de profundidad n construye, en su forma canónica, n marcos de activación consecutivos en el stack. Para factorial(20), esto implicaría hasta 20 marcos de activación. Si cada marco ocupa aproximadamente 32 bytes (dirección de retorno + RBP guardado + RBX guardado + alineación), el consumo estimado se aproximaría a 640 bytes de stack en condiciones sin optimización. En contraste, la implementación iterativa podría consumir 0 bytes adicionales de stack si el compilador logra mantener las variables en los registros RAX y RDI sin spills. Los valores específicos —tanto el tamaño exacto de cada marco como la presencia de spills— dependerán del nivel de optimización del compilador y serán verificados mediante inspección del ASM generado y medición de RSP en ejecución. Esta diferencia esperada $O(n)$ vs $O(1)$ en uso de stack constituye el objeto cuantificable central de MEMARCH.

E. Pipeline

El pipeline de instrucciones es la técnica microarquitectónica que permite que múltiples instrucciones se ejecuten simultáneamente en diferentes etapas de procesamiento, incrementando el throughput del procesador sin necesidad de incrementar la frecuencia de reloj. El pipeline clásico de cinco etapas (IF, ID, EX, MEM, WB) fue introducido por el procesador MIPS R2000 [2]; los procesadores modernos de la familia Intel Core y AMD Zen implementan pipelines de 14 a 20 etapas con ejecución fuera de orden.

El pipeline introduce tres categorías de riesgos (hazards) que reducen su eficiencia:

- Hazards estructurales: conflicto en el uso de un recurso hardware compartido por múltiples instrucciones simultáneas.
- Hazards de datos: una instrucción requiere el resultado de otra que aún no ha completado su etapa EX.
- Hazards de control: una instrucción de salto (CALL, JMP, RET) invalida las instrucciones ya captadas bajo el flujo secuencial anterior.

Las instrucciones CALL y RET son especialmente adversas para el pipeline. CALL interrumpe el flujo secuencial de instrucciones, provocando un flush (vaciado) parcial del pipeline hasta que la dirección destino se resuelve. RET depende del Return Address Stack para predicción. Cuando la pila RAS se desborda (profundidad recursiva mayor que ~16 entradas, capacidad dependiente del modelo

de procesador), la predicción de RET se degrada y pueden producirse penalizaciones del orden de 15–25 ciclos por mispredicción en microarquitecturas modernas [3]. Estos valores son referenciales y dependen del modelo específico de CPU; el comportamiento real en el equipo empleado por MEMARCH se contrastará mediante contadores de hardware (branch-misses) antes de presentarlos como conclusión propia.

La implementación iterativa, al sustituir CALL/RET por JNZ (jump if not zero) o JLE (jump if less or equal), tiende a favorecer al predictor de saltos, puesto que los bucles exhiben patrones altamente regulares (taken $N-1$ veces, not-taken 1 vez) que los predictores modernos suelen aprender con alta precisión tras pocas iteraciones. La magnitud real de esta reducción de penalización en MEMARCH se verificará experimentalmente mediante perf (branch-misses) y no se asume a priori para todos los valores de n .

F. Entrada/Salida

En el modelo de Von Neumann clásico, los dispositivos de entrada/salida (E/S) interactúan con la CPU a través del bus del sistema, compartiendo el mismo canal de comunicación que la memoria principal. Los mecanismos modernos de E/S —interrupciones, acceso directo a memoria (DMA), puertos mapeados en memoria (MMIO)— son extensiones que mitigan el cuello de botella original al desacoplar la CPU de la gestión directa de dispositivos [7].

En el contexto específico de MEMARCH, la E/S tiene un papel instrumental: el programa principal en C utiliza llamadas al sistema de Linux (write, a través de printf) para reportar resultados de benchmark. Estas llamadas al sistema implican transiciones modo-usuario a modo-kernel que introducen latencia no despreciable (~1,000 ciclos en procesadores modernos). Por esta razón, las mediciones de tiempo se realizan con `clock_gettime(CLOCK_MONOTONIC_RAW)` rodeando únicamente las llamadas ASM, excluyendo las operaciones de E/S del intervalo medido, para aislar el comportamiento del algoritmo del overhead del sistema operativo.

Adicionalmente, MEMARCH utilizará el subsistema de contadores de hardware vía `perf_event_open` como mecanismo de E/S especializado: permite leer contadores del procesador (cycles, instructions, cache-misses, branch-misses) sin intervención del bus principal, ya que los contadores son registros internos del procesador accesibles mediante instrucciones RDPMC, constituyendo un mecanismo de observabilidad de bajo overhead, importante en el tune-up de aplicaciones a bajo nivel.

G. Multiprocesamiento

El multiprocesamiento es la organización de sistemas que incorporan múltiples unidades de procesamiento capaces de ejecutar instrucciones de forma concurrente. Los procesadores modernos de escritorio y servidor son todos multiprocesadores en el sentido de que integran múltiples núcleos (cores) en un mismo die de silicio, cada uno con su pipeline, su caché L1/L2 privada y acceso compartido a la caché L3 y la DRAM [2].

Los sistemas multiprocesador introducen el problema de la coherencia de caché: cuando múltiples núcleos tienen copias del mismo bloque de memoria en sus cachés privadas, es necesario un protocolo que garantice que todas las lecturas observen el valor más reciente escrito. El protocolo MESI (Modified, Exclusive, Shared, Invalid) y sus variantes (MOESI, MESIF) resuelven este problema mediante una máquina de estados por bloque de caché [8].

Para MEMARCH, el multiprocesamiento introduce una consideración experimental crítica: la afinidad de proceso. Para obtener mediciones reproducibles y comparables, el proceso de benchmark debe ejecutarse en un único núcleo específico, evitando migraciones que invalidarían la caché L1/L2 y contaminarían las mediciones de tiempo. Esto se logra mediante la llamada al sistema `sched_setaffinity()` o el comando `taskset` de Linux, que fija el proceso a un núcleo específico antes de iniciar la instrumentación.

En un escenario de multiprocesamiento paralelo, la recursión presentaría desafíos adicionales relacionados con el stack: cada hilo (thread) en un sistema multihilo tiene su propio stack privado, de tamaño limitado (típicamente 8 MB en Linux, configurable vía ulimit). Una recursión profunda en múltiples hilos simultáneos puede agotar la memoria de stack de manera colectiva mucho antes que en un escenario monohilo, haciendo que la diferencia $O(n)$ vs $O(1)$ sea arquitectónicamente determinante en sistemas de alta concurrencia.

H. Métricas específicas de validación y control de variaciones experimentales

Dado que la hipótesis central H1 se enuncia en términos de magnitudes medibles ($O(n)$ vs $O(1)$ en uso de stack y diferencias en contadores de hardware), resulta indispensable explicitar el conjunto de métricas con que se evaluará y las estrategias de control de variaciones derivadas del compilador y del entorno. Esta subsección distingue el "ambiente de ejecución" descrito en secciones previas (qué sistema operativo, qué compilador, qué CPU) del "modelo del sistema" propio de MEMARCH, entendido como el conjunto de variables independientes manipuladas, las variables dependientes medidas y los controles que aíslan el efecto causal a verificar.

1) Métricas primarias (variables dependientes):

- Profundidad máxima de stack consumida, medida por delta de RSP capturado in situ desde la propia rutina ASM (no únicamente desde C, para evitar restauración antes de la lectura).
- Instrucciones retiradas (perf: instructions) y ciclos consumidos (perf: cycles), con cálculo de IPC (instructions per cycle) como métrica derivada.
- Branch-misses y branch-instructions (perf), para verificar la hipótesis de degradación del RAS con recursión profunda.
- Cache-misses en L1/LLC (perf: L1-dcache-load-misses, LLC-load-misses) para detectar efectos de localidad asociados al stack.

•Tiempo de ejecución con clock_gettime(CLOCK_MONOTONIC_RAW) en torno al kernel del benchmark, repetido $N = 10^6$ veces.

2) Variables independientes manipuladas:

- Paradigma de implementación (recursivo / iterativo / memoizado): factor principal.
- Tamaño de entrada n (recorrido en una rejilla logarítmica para cubrir comportamiento dentro y fuera de la capacidad del RAS).
- Nivel de optimización del compilador (-O0 y -O2 como mínimo; eventualmente -O3) para distinguir comportamiento "canónico" frente a comportamiento "optimizado por GCC".

3) Control de variaciones derivadas del compilador y del entorno:

- Compilador: reportar versión exacta de GCC y opciones completas en cada corrida; conservar el .s generado (gcc -S -masm=intel) como evidencia auditable de las transformaciones aplicadas. Las rutinas ASM escritas a mano sirven como "ground truth" frente a la salida del compilador.
- Afinidad de núcleo: fijar el proceso a un único core físico vía taskset / sched_setaffinity para evitar migraciones que invaliden la caché L1/L2 y contaminen las mediciones.
- Governor de CPU: configurar en "performance" para evitar dynamic frequency scaling (DVFS) que distorsionaría las mediciones de tiempo entre corridas.

- Calentamiento de caché: descartar las primeras k iteraciones de cada serie (warm-up) y reportar únicamente el régimen estacionario en su caso.
- WSL2: registrar la versión del kernel y la edición de Windows host; las mediciones de perf en WSL2 son válidas para conteo de eventos por proceso, pero se documentará cualquier limitación detectada en el acceso al PMU desde el subsistema.

La distinción metodológica entre las afirmaciones derivadas de la literatura (que en este avance se presentan como referencias y expectativas razonables) y las afirmaciones derivadas de la medición propia (que se presentarán en los avances subsecuentes una vez completada la fase experimental) es deliberada. El presente documento describe el marco teórico y el plan de medición; las conclusiones cuantitativas sobre el comportamiento real del compilador, del predictor de saltos y del pipeline en el entorno MEMARCH se reportarán como evidencia experimental en las dos semanas siguientes del proyecto.

VI. DIAGRAMA CONCEPTUAL DE ARQUITECTURA PROPUESTA

La Figura 1 presenta el diagrama conceptual de la arquitectura experimental de MEMARCH. El sistema se organiza en cuatro capas de abstracción: el algoritmo fuente (capa de lenguaje), la integración C+ASM (capa de compilación), el entorno de ejecución x86-64 (capa microarquitectónica) y la capa de observabilidad (instrumentación).

FIGURA 1 — Arquitectura Experimental MEMARCH
CAPA 1 — Algoritmos Fuente Factorial Recursivo Factorial Iterativo Fibonacci + Memoización <i>O(n) stack frames · O(1) stack frames · O(n) con memo[]</i>
CAPA 2 — Implementación e Integración (C + NASM) Rutinas ASM (NASM x86-64): factorial_rec.asm · factorial_iter.asm · fib_memo.asm Programa Principal (C puro): main.c — benchmark + instrumentación + reporte
CAPA 3 — Entorno de Ejecución x86-64 CPU (Pipeline · ALU · RAS · Predictor) · Stack RSP/RBP (Frames · ABI) Caché L1/L2/L3 (Localidad · Miss rate) · DRAM (Cuello de botella VN)
CAPA 4 — Observabilidad e Instrumentación perf stat: cycles · instructions · cache-misses · branch-misses clock_gettime CLOCK_MONOTONIC_RAW · objdump / GCC -S · TCE analysis

Figura 1. Arquitectura experimental MEMARCH: cuatro capas de abstracción desde el algoritmo fuente hasta la instrumentación de hardware. Fuente: elaboración propia.

VII. REFERENCIAS

[1] J. von Neumann, "First Draft of a Report on the EDVAC," Moore School of Electrical Engineering, University of Pennsylvania, Tech. Rep., 1945. Reprinted in *IEEE Annals of the History of Computing*, vol. 15, no. 4, pp. 27–75, 1993.

[2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2020.

[3] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer's Manual*, Combined Volumes 1, 2A, 2B, 3A, 3B, 3C, 3D, and 4. Santa Clara, CA, USA: Intel Corp., 2024. [Online]. Available: <https://www.intel.com/sdm>

[4] R. Bryant and D. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2016.

- [5] J. W. Backus, "Can Programming Be Liberated from the Von Neumann Style? A Functional Style and Its Algebra of Programs," *Communications of the ACM*, vol. 21, no. 8, pp. 613–641, Aug. 1978. doi: 10.1145/359576.359579.
- [6] M. Matz, J. Hubicka, A. Jaeger, and M. Mitchell, *System V Application Binary Interface: AMD64 Architecture Processor Supplement*, Version 1.0, 2013. [Online]. Available: https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf
- [7] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2015.
- [8] M. Papamarcos and J. Patel, "A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories," in *Proc. 11th Annual Int. Symp. on Computer Architecture (ISCA)*, Ann Arbor, MI, USA, 1984, pp. 348–354. doi: 10.1145/800015.808204.
- [9] T. M. Conte and W. W. Hwu, "Benchmark Characterization," *IEEE Computer*, vol. 24, no. 1, pp. 48–56, Jan. 1991. doi: 10.1109/2.67193.
- [10] A. Fog, *Optimizing Software in C++: An Optimization Guide for Windows, Linux, and Mac Platforms*. Copenhagen University College of Engineering, 2024. [Online]. Available: <https://www.agner.org/optimize/>
- [11] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.
- [12] U. Drepper, "What Every Programmer Should Know About Memory," Red Hat, Inc., Tech. Rep., 2007. [Online]. Available: <https://www.akkadia.org/drepper/cpumemory.pdf>